

OCP Graph Agent

A knowledge graph of OCP's business, with agents that read messy Excel files, follow world events to OCP's products and plants, and explain why numbers move, without ever inventing a number.

Author	Date	Status	Code
Abdelghafour Lahrache	10 October 2026	Working prototype, not validated by OCP finance	github.com/Abdelghafour2001/graph-stuff

Summary

- **Problem.** OCP's P&L depends on volatile prices, costs and volumes that arrive as Excel files with no common template, plus market databases and news. Computing and explaining it is slow and hard to trace.
- **Approach.** A Neo4j graph holds the business (org, sites, products, recipes, metrics, routes, vocabulary) and links every source to it: 739 workbook sheets, 37,627 Argus articles, 2,837 verified price assessments, 757 events, 61 OCP assets, 20 supply-chain lanes. Agents use this graph through tools; deterministic code does all arithmetic.
- **Agents.** An orchestrator routes each question to three specialists (impact analyst, workbook agent, news agent). A deterministic Reflector rejects any number that does not come from a tool result.
- **What works today.** Extraction specs that pass 12 checks (6 of 8 hard sheets on a small model, one agreeing with the old hand-written pipeline at 99%); a variance diagnosis that finds causes passing through from the previous month (top-1 0.70 vs 0.18 for the naive method, on a synthetic market); an exposure walk from events to OCP products; structure discovery and a P&L engine for branch files with no template.
- **What it cannot do yet.** Size OCP's gains or losses: the graph has no OCP volumes, costs or results. The supply-chain lanes and input ratios are public knowledge or illustrative until OCP confirms them.

1 Purpose of the research

The problem as OCP lives it

OCP Group has many branches and business units, each with its own products and processes. The group regularly asks its branches for numbers (budget, rolling forecast, landing estimate). Each branch answers with its own spreadsheet: volumes by product and site, selling prices, consumption of rock, sulphur, ammonia and energy, input prices, freight, FX and fixed costs. There is no group template and no shared structure, and the layout can change from one cycle to the next.

From the day a submission is sent, it ages. Sulphur triples, a strait closes, a plant stops, a tariff lands, and nothing tells the group which assumptions are now wrong, in which branch, or by how much. In H1 2026, public reporting described sulphur prices roughly tripling while phosphate fertilizer prices rose about 20%, and OCP's EBITDA falling 28.5% (Fertilizer Daily, Morocco World News). That is the kind of movement the system should explain, driver by driver.

Goals

1. **Understand the business:** plants, products, processes, countries, markets, and the vocabulary people use (French, English, internal abbreviations).
2. **Answer by retrieving relations:** what is made where and from what, prices, forecasts, and the events that affect them.
3. **Explain changes:** why a margin or cost moved between two periods, with every step traceable to a source.
4. **Compute the P&L eventually,** from a formula graph that adapts as sources and processes change.

Rules that do not bend

Rule	How it is enforced
The LLM never computes or invents numbers	Deterministic tools compute; the Reflector rejects any number ≥ 10 in an answer that is not in a tool result (within 1%); <code>calc</code> for derived numbers
Every claim cites evidence	Concept ids, file / sheet / cell, article ids; answers without article ids are sent back
The curated core changes only through review	Agents write to review queues (terms, specs, entity merges, diagnoses); humans approve in the UI
History is never overwritten	Versioned specs and diagnoses; branch submissions planned as versioned assumptions

2 Research: techniques and where they come from

The design borrows from published work on spreadsheet understanding, graph-based agents, root-cause analysis, and from how other industries already explain P&L. Each line below says what was taken and where it lives in the code.

2.1 Spreadsheets

Work	Technique	What we took
SheetCompass (arXiv 2608.14452)	Hierarchical table → column graph; Explorer / Programmer / Reflector agents	Workbook graph, role split, Reflector loop
SpreadsheetAgent (arXiv 2604.12282)	Reads local regions; YAML layout description; verification module	Extraction spec as YAML; read regions through tools; verify before trusting
Beyond Rows to Reasoning (arXiv 2603.06503)	Agentic retrieval over multi-sheet workbooks; tool calls beat dumping the sheet	Tools <code>describe_sheet</code> and <code>read_range</code> instead of whole sheets
SpreadsheetLLM (arXiv 2407.09025)	Skeleton and structural anchors, ~25× compression	Sheet skeletons shown to the agent
Auto-Tables (arXiv 2307.14565)	Closed operator set: ffill, stack, subtitle, transpose, wide_to_long, ...	The agent chooses operators, never writes free code: specs are deterministic and replayable
pycel, excel-grapher	Formula → dependency graph	<code>formula_graph.py</code> : 395,617 formulas of one S&P file reduce to 67 rules

2.2 Graphs, agents, events

Work	Technique	What we took
Agentic reasoning graphs (GraphBit, A2RAG, ...)	Deterministic orchestration; keep provenance	Deterministic tools and checks around the LLM; events always linked to their articles
Graph-PReFLexOR (arXiv 2502.13025)	Self-growing graph, structure emerges	Only for the news layer; the finance core stays curated
Helicase (arXiv 2605.26835)	Supply-chain KG with uncertainty per fact	Status and confidence on self-grown edges (events, merges, lanes)
Graphiti / Zep	Bi-temporal KG on Neo4j	"As of" queries on publication dates; candidate for the events layer
Supply-chain intelligence POCs	World events → supplier / plant / revenue risk	Shape of the exposure walk (section 6.4)

2.3 Root-cause analysis

Work	Technique	What we took
KG-guided RAG for RCA (arXiv 2608.11277)	Deterministic evidence vs a healthy baseline, KG candidate scores, case retrieval on z-vectors, LLM as reranker with strict JSON, window votes	The whole shape of <code>diagnose_variance</code> , mapped to P&L drivers; injected faults as ground truth
Causal-LLM (budget variance)	Variance threshold triggers causal discovery on a financial causal KG	Our <code>DEPENDS_ON</code> edges as the causal prior
Cascading disruptions (arXiv 2605.06411)	Shocks propagate from gas and fertilizer inputs downstream	Propagation along <code>MADE_FROM</code> ; explain-away rule

Work	Technique	What we took
ConflictQA / XoT (arXiv 2604.11209)	LLMs pick the wrong side when KG and text disagree	Show both sides; never let the LLM settle a conflict silently
Case-based reasoning with LLMs	Structured cases beat embedded text	Analog months as z-vectors, labelled by human review

2.4 Practice in other industries

Practice	Where	What we took
P&L explain with an unexplained residual	Trading desks (FRTB attribution tests)	Mark each plan to market and split the change by driver; the unexplained share measures what the graph is missing
Published sensitivities	Mining (e.g. BHP: EBITDA per US\$1 price move)	Sensitivity of each line to each input, computed from the formula graph
Indicative margins from benchmark prices	Refining	$\text{proxy_margin} = \text{product price} - \sum \text{ratio} \times \text{input price}$, once ratios are validated
Ontology as a digital twin	Palantir Foundry	The Neo4j graph plays the ontology role; simulation stays deterministic and auditable
Driver-based and agentic planning	FP&A tools	Store branch submissions as drivers; events re-trigger the calculation

Methodology warnings kept in mind: a news event and a price move in the same window are not proof of cause; use "as of" dates to avoid look-ahead; report the evidence, not a causal claim.

3 The data landscape

3.1 What already exists at OCP market intelligence

Source	Size	What it is
<code>corporate_data.argus_news</code>	37,627	Argus news 2017-09 → 2026-06, full text
<code>corporate_data.sp_documents</code>	45,936	S&P documents
<code>market_knowledge_data.metrics</code>	5,833,952	Normalised long-format values from ingested Excels
Views <code>v_prices</code> / <code>v_supply</code> / <code>v_demand</code> / <code>v_trades</code> / <code>v_costs</code>	352k / 1.99M / 590k / 1.23M / 53k	Materialised views on metrics
<code>ref_entities</code> , products, countries, regions	13,059 / 110 / 284 / 128	Referentials, including OCP sites and competitors
Excel pipeline	56 hand-written casters	One script per layout; 875 of 1,080 sheets skipped as "no extractor match"

3.2 Why the Excel files are hard (40 workbooks, 739 sheets profiled)

Trait	Sheets	Why it hurts
Header not on row 1	587 (79%)	Notices, logos and titles on top
Multi-row headers	345 (47%)	Unit row + product row + incoterm row
≥ 3 tables stacked	297 (40%)	Several tables in one sheet
Contain formulas	441 (60%)	Total rows mixed with leaf rows: reading everything double-counts
No detectable header	132	Dashboards, charts, pivots
Contents / Index sheets	32	Useful: they describe the other sheets (262 descriptions reused)

3.3 Findings on the sample provider files

- **Forecasts are marked only by cell format** (`mmm-yy "f"`): a value reader cannot tell forecast from history, which would leak the future into every "as of" query. The executor now reads the format and tags each row `forecast` or `actual`.
- **Layouts drift between editions** by one row; approved specs now re-anchor on header text and refuse to load when the anchor is gone.
- **"No market" is written three ways** (0, NM, n/a): treated as missing per provider.
- **Source errors flagged, not fixed**: a `1Q25` that should read `1Q26`, a Mar-2018 row inside 2019 (caught by the new period-order check).
- **The old pipeline stored serialised pandas objects as values**: 10,440 corrupted values in 4 S&P sheets, plus 162,054 'nan' rows.
- **The vessel tracker** is record-shaped (one voyage per row) and needs its own extractor; it would measure the Hormuz exposure the graph now assumes.

4 The knowledge graph

The graph is organised in layers around shared `Concept` nodes. A concept id (`dap` , `sulfur` , `country_morocco` , `hormuz`) is the join key between layers: a sheet, an article, an event, a price, a plant record and a supply lane all point to the same concepts, which is what lets an agent follow a question across sources.

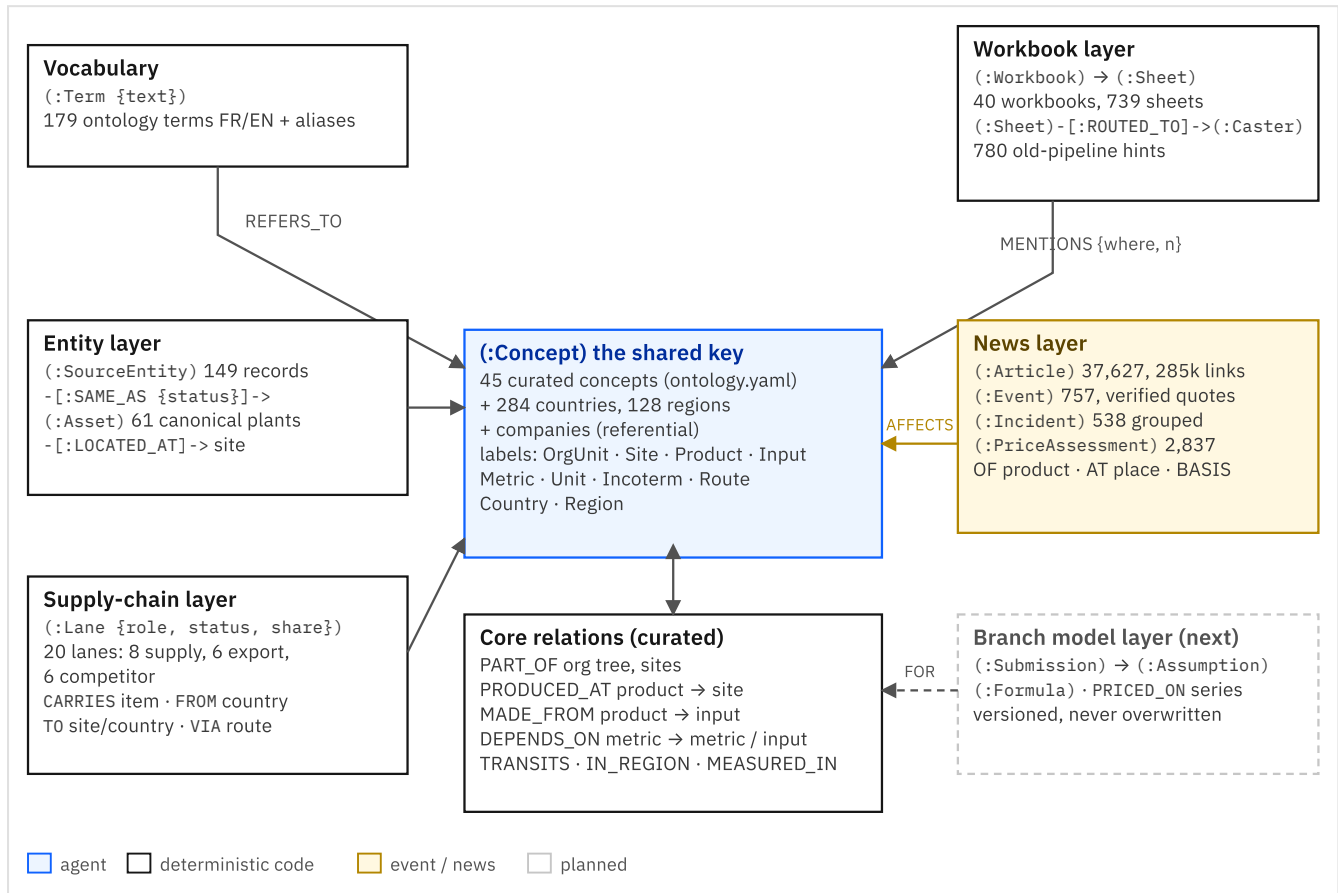


Figure 1. Graph layers. Counts are from the last full load of the market-intel data.

4.1 The ontology

The curated core lives in one human-edited file, `knowledge/ontology.yaml` : 45 concepts with a name, a definition and their terms, and 54 typed relations. It was seeded by hand from OCP's public structure and the domain (phosphate value chain, inputs, metrics) and must be validated by finance and BU controllers. Every concept needs a name, a definition and at least one term; every relation must use a known type and point to known concepts, or the load fails.

Kind	Count	Examples
product	10	phosphate_rock, sulfuric_acid, phosphoric_acid, DAP, MAP, TSP, NPS, urea, potash, SSP
site	6	Khouribga, Benguerir, Youssoufia, Boucraa (mining); Jorf Lasfar, Safi (chemicals)
org_unit	3	ocp_group, mining, nutricrops
input	4	sulfur, ammonia, energy, freight
metric	6	revenue, cogs, cash_cost, gross_margin, ebitda, transfer_price
unit · incoterm	5 · 6	tonne, t P2O5, BPL, USD, MAD · fob, cfr, cif, fca, cpt/cip, exw
route	5	Hormuz, Bab el-Mandeb, Suez, Cape of Good Hope, Turkish Straits

Relation	Count	Meaning
MADE_FROM	16	product → product or input (the recipes)
PRODUCED_AT	13	product → site
DEPENDS_ON	9	metric → metric or input (the causal prior for variance)
PART_OF	8	org tree: sites → business unit → group
MEASURED_IN · TRANSITS · IN_REGION	5 · 3 · imported	units; inputs whose world trade passes a route; country → region

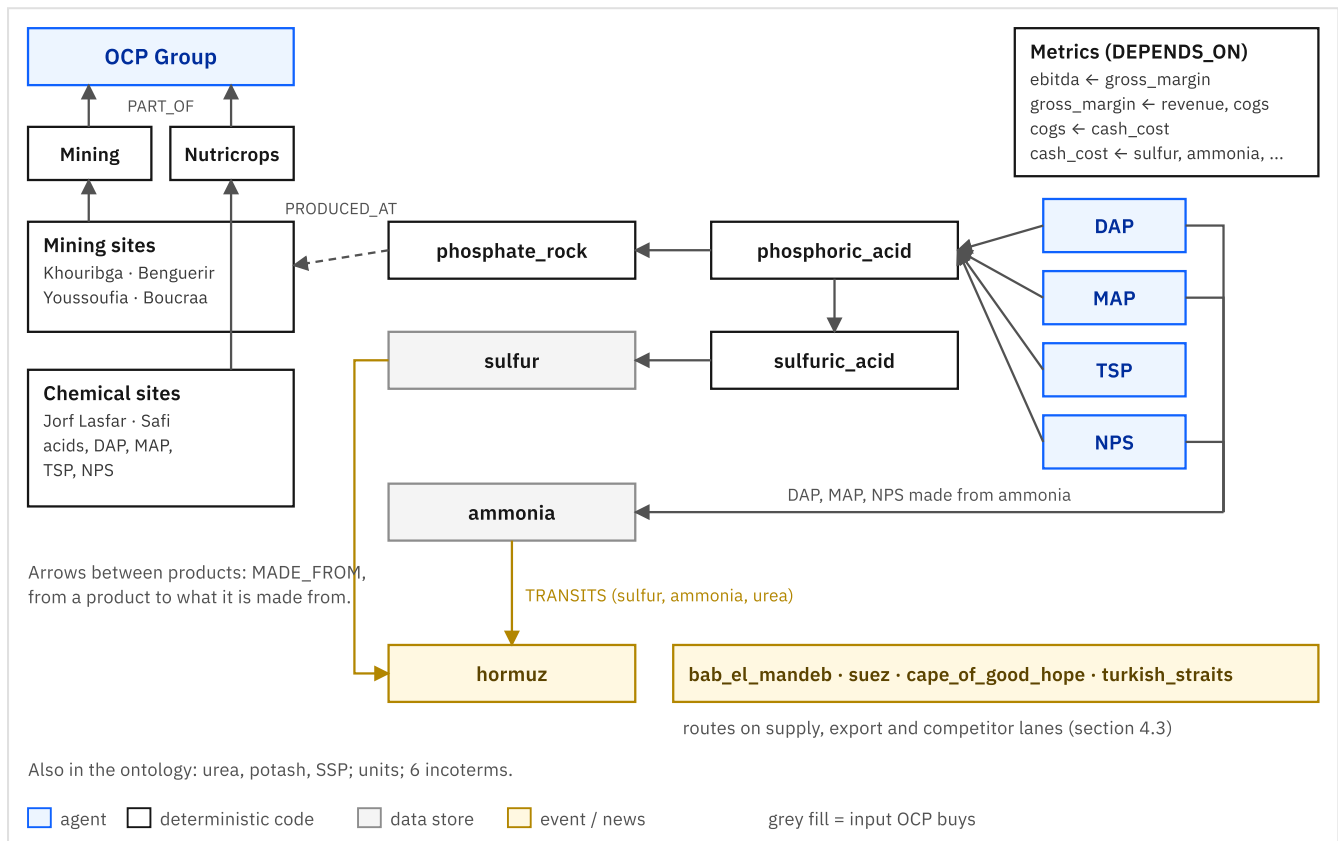


Figure 2. The curated core as instances: who makes what, from what, and which metrics depend on which inputs.

4.2 Vocabulary

People say "ACP", "acide phosphorique", "phosacid" and mean the same thing. Each concept carries its terms (179 in the ontology), the referential import adds country, region and product aliases, and approved proposals land in `term_additions.yaml`. When the agent meets an unknown word it calls `propose_term`, which writes to a review queue and never to the graph. A test now guarantees that no ontology term also names a country, region or company, because prices and events are matched to concepts by name and the first match wins.

4.3 How the graph is built

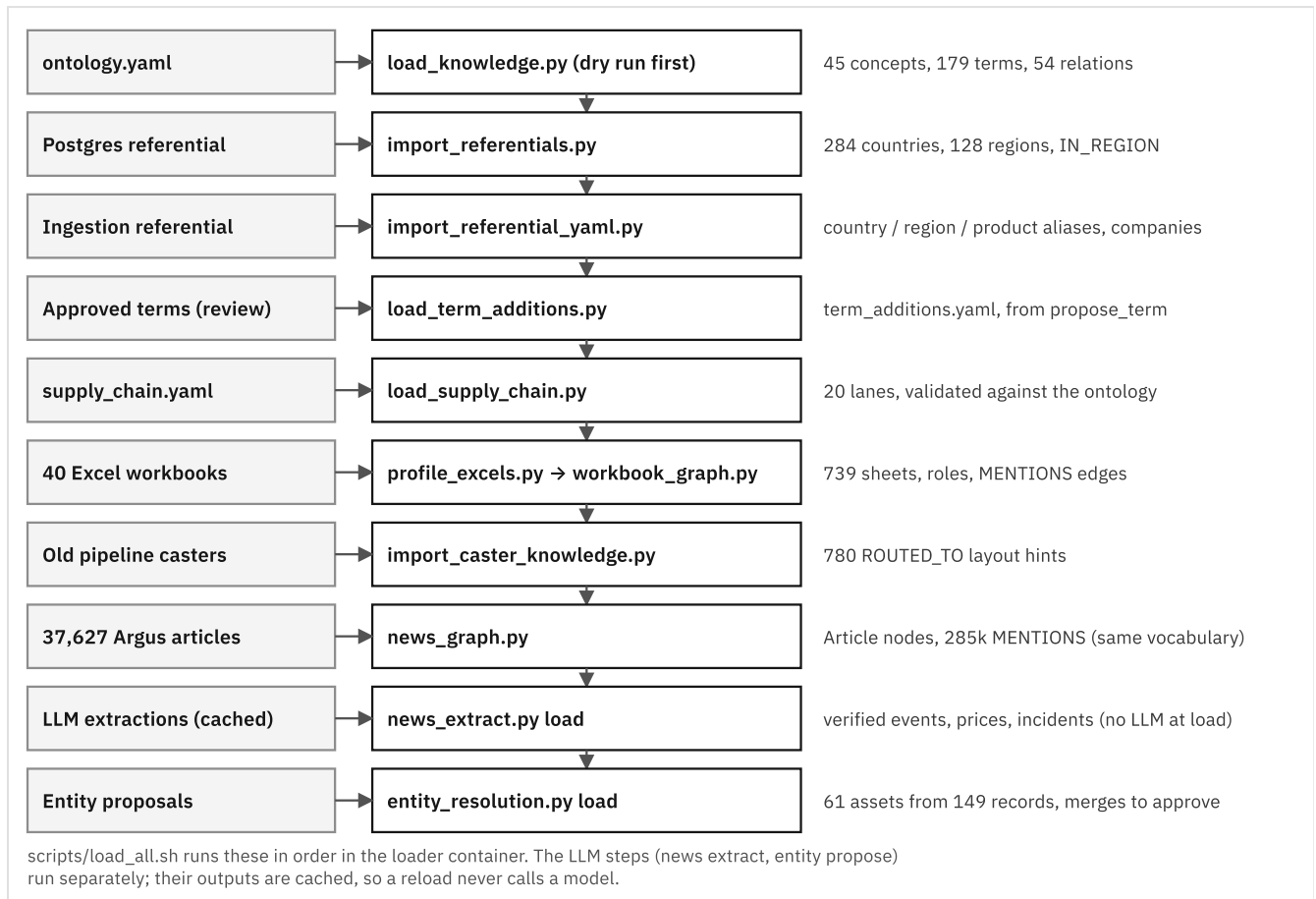


Figure 3. Load order (`scripts/load_all.sh`). The ontology reload shows a dry-run diff first, including edges other loaders would lose.

- **Workbook layer.** A profiler reads each sheet's layout (header rows, blocks, formulas, merged cells); **MENTIONS** edges link sheets to the concepts in their headers, rows and context; the old pipeline's casters are kept as layout hints.
- **News layer.** All articles are linked to concepts with the same vocabulary. A small model extracted events and price assessments once (cached); at load, an item is kept only if its quote is found verbatim, both prices appear in the quote, and the price is within 2× of its series median. Events reported by several articles are grouped into incidents.
- **Entity layer.** 149 OCP-related source records (spellings like *Laayoune* / *Layoune* / *La youne*) are clustered by the model, checked by code, and become 61 canonical assets; merges stay "proposed" until approved.
- **Supply-chain layer.** 20 lanes say what OCP buys and from where, where it sells, and who competes with it, with the shipping routes in between. All are marked *public knowledge, not confirmed by OCP*, with no shares, until procurement and sales confirm them.

5 The agentic architecture

An orchestrator talks to the user and routes the work to three specialists, each with its own short prompt, its own tools and, optionally, its own model. Specialists are exposed to the orchestrator as tools ("agent as tool"), so no framework is needed. Calls made in the same turn run in parallel.

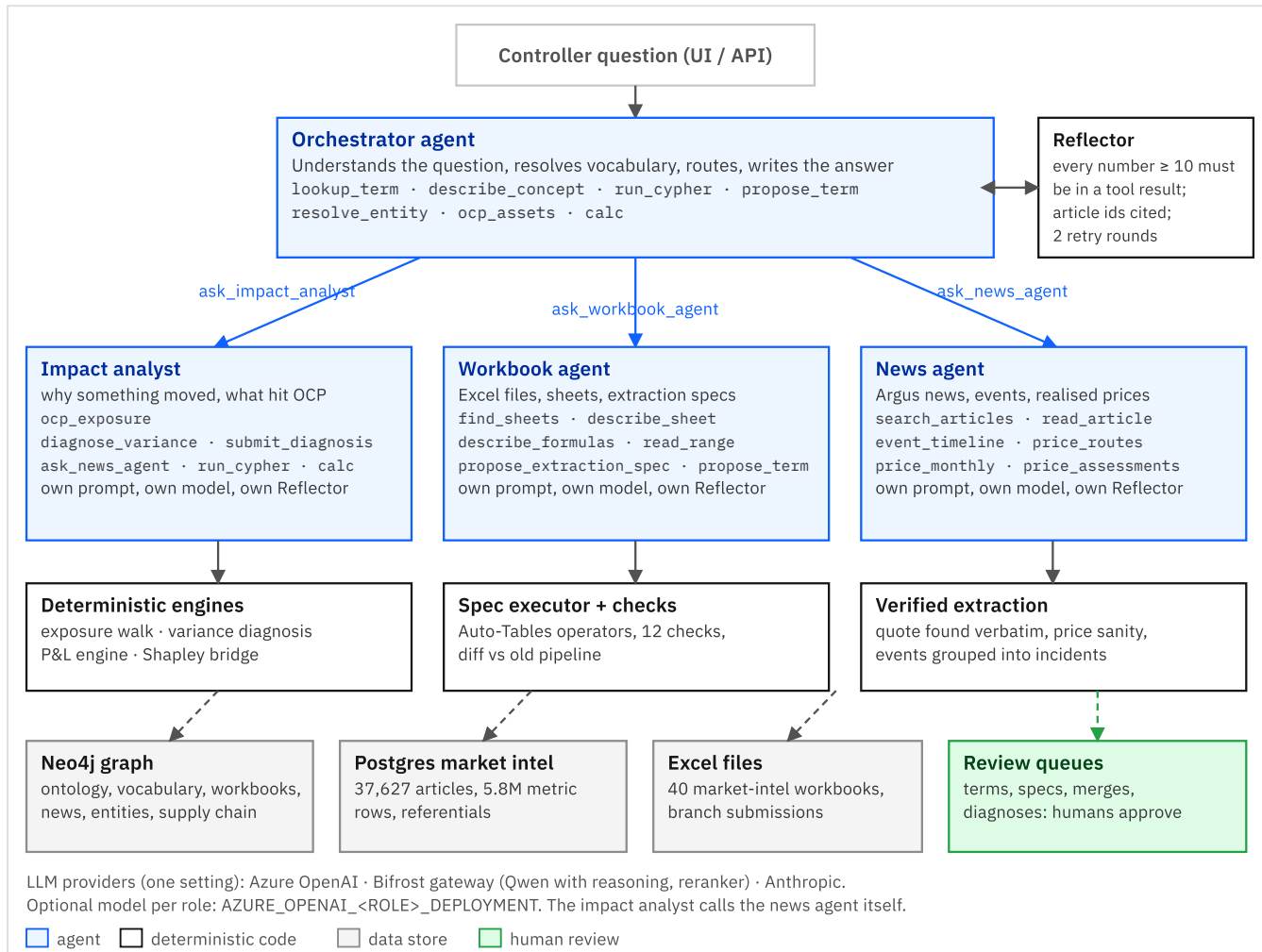


Figure 4. Agents, the tools each one may use, the deterministic engines behind them and the data they read.

Role	Tools	Job
Orchestrator (10 tools, ~500 words of instructions)	<code>lookup_term</code> , <code>describe_concept</code> , <code>run_cypher</code> , <code>propose_term</code> , <code>resolve_entity</code> , <code>ocp_assets</code> , <code>calc</code> + 3 specialists	Vocabulary, plants and companies, routing, the final answer
Impact analyst	<code>ocp_exposure</code> , <code>diagnose_variance</code> , <code>submit_diagnosis</code> , <code>ask_news_agent</code> , lookups, <code>calc</code>	Why a price, margin or OCP's position moved; what hit OCP in a period
Workbook agent	<code>find_sheets</code> , <code>describe_sheet</code> , <code>describe_formulas</code> , <code>read_range</code> , <code>propose_extraction_spec</code> , <code>propose_term</code>	Where a fact lives in the Excel files; checked extraction specs
News agent	<code>search_articles</code> , <code>read_article</code> , <code>event_timeline</code> , <code>price_routes</code> , <code>price_monthly</code> , <code>price_assessments</code>	Events, articles and realised Argus prices, all "as of" a date

How numbers stay honest across agents

- A specialist returns its answer *and* the raw tool results. The orchestrator's Reflector checks the final answer against those results, so a number cannot drift as it passes from one model to another.

- Every agent run (orchestrator and each specialist) has its own Reflector: numbers ≥ 10 must appear in a tool result within 1%; if tools returned article ids, at least one must be cited; failures go back to the model twice, then are appended to the answer.
- The agent never sees raw series when it ranks causes: it gets a compact evidence report and must answer in strict JSON that code checks.

Models and providers

One setting switches between Azure OpenAI, the Bifrost gateway (an OpenAI-compatible gateway in front of a self-hosted Qwen model with reasoning, plus a reranker and embeddings) and the Anthropic API. Each role can use its own deployment (`AZURE_OPENAI_IMPACT_DEPLOYMENT` , ...). Bulk extraction uses a cheaper model; agents use the strongest available. On Bifrost, the Chat Completions API is used (not Responses), Qwen's thinking is switched per role, and `<think>` blocks are stripped before any answer or JSON is read. The reranker breaks ties when mapping branch labels to concepts.

6 The current flows

6.1 Excel files into data: checked extraction specs

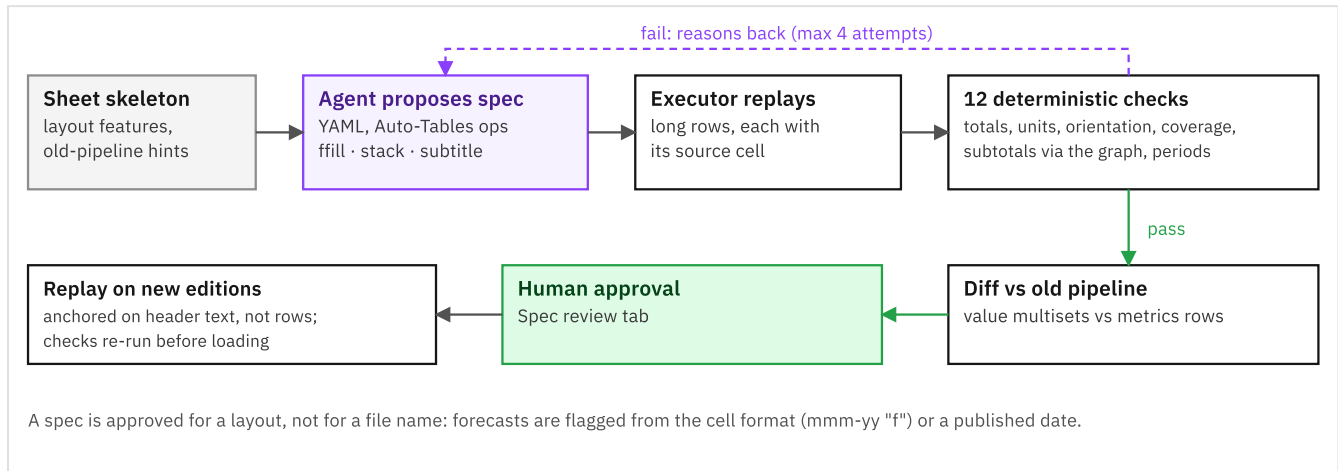


Figure 5. The agent proposes a spec in a closed operator vocabulary; code replays it and checks it; failures go back with reasons.

Check	Catches
values_parse · header_is_text · labels_present	wrong header or label rows
covers_whole_table	truncated ranges
unit_evidence	unit not supported by the preamble (<i>000 tonnes</i> → kt, not t)
corner_orientation	<i>Importers Exporters</i> corner cell read the wrong way
no_total_rows / columns, dropped_ * justified	double counting, or dropping real data to make checksums pass
no_subtotal_labels	region subtotal rows, found through the graph's <code>IN_REGION</code> edges
status_marked · column / row periods in order	forecasts read as actuals; dates out of order

Results. On 8 hard sheets with a small model (gpt-4.1-mini), 6 pass every check; each rejection is a real, explained issue. Where the old pipeline has data, the diff is decisive: on the Argus Price Forecast sheet the agent's spec agrees with the hand-written caster (100% of our values in the database, 99.3% of the database's in ours). Four passing sheets were never extracted before: net new data. The first unchecked spec, by contrast, had flipped the trade direction, double-counted a total row and mis-read the unit, which is why the checks exist.

6.2 News into events and prices

Argus daily reports give price assessments (product, place, incoterm, low/high, quote); war-and-fertilizer articles give events (type, date, direction, channel, affected concepts). Only verified items are loaded, so every number the news agent quotes has a verbatim source sentence and an article id. Example output of the news agent: realised DAP prices rose from Feb to Jun 2026 by +27% China fob, +35% India cfr and about +24% Morocco fob (thin data), with the first Hormuz events dated 28 February 2026.

6.3 Why did a metric move? The variance diagnosis

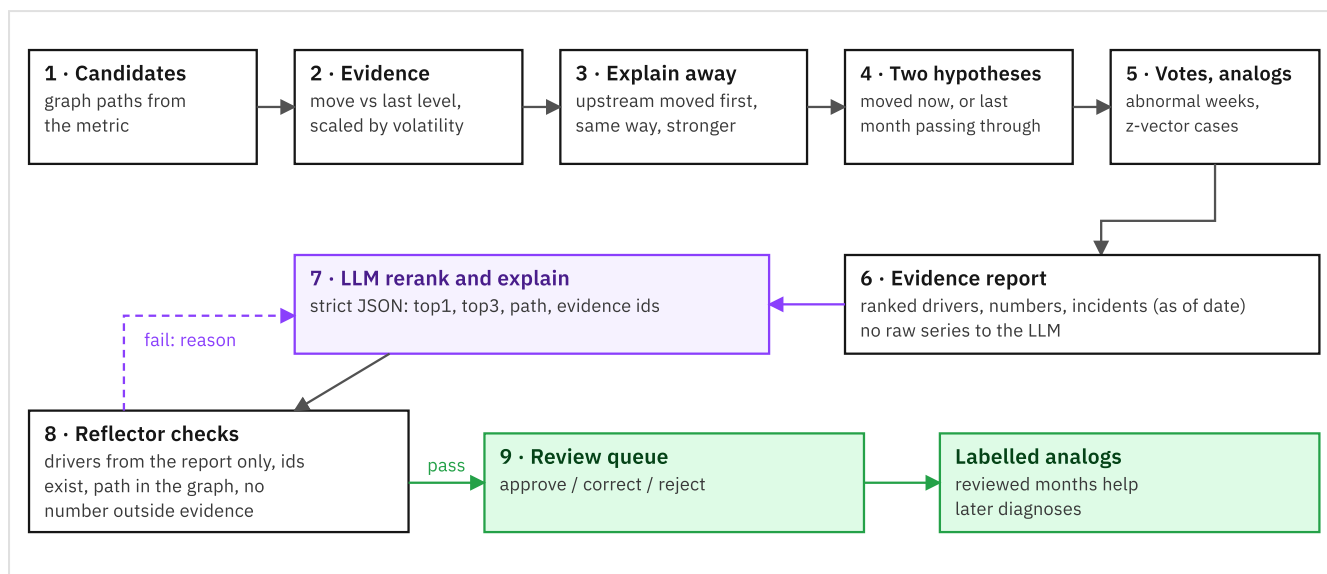


Figure 6. The LLM is a reranker and explainer, not a diagnostician working from raw numbers.

The first version, following the RCA paper literally, lost to a naive "largest % change" baseline. Three changes fixed it: measure moves from the last level scaled by each driver's volatility; let the graph *explain away* a downstream move when an upstream driver moved first, the same way and at least as strongly; and test two hypotheses per driver, "moved now" and "moved last month and is passing through".

Case (synthetic market, 3 seeds, 18 months)	Top-1	Top-3	MRR	Naive top-1
Shock in the month diagnosed (648 cases)	0.90	0.99	0.95	0.92
Shock one month earlier, passing through (416 cases)	0.70	0.93	0.82	0.18

The method is on par with the naive baseline when the cause moves inside the month; its value is the second row, the question controllers actually ask. Analogs add nothing yet: they need reviewed diagnoses, which the review queue now collects. The synthetic market is a test bench; real labelled episodes (2026 Hormuz) are next.

6.4 What hit OCP? The exposure walk

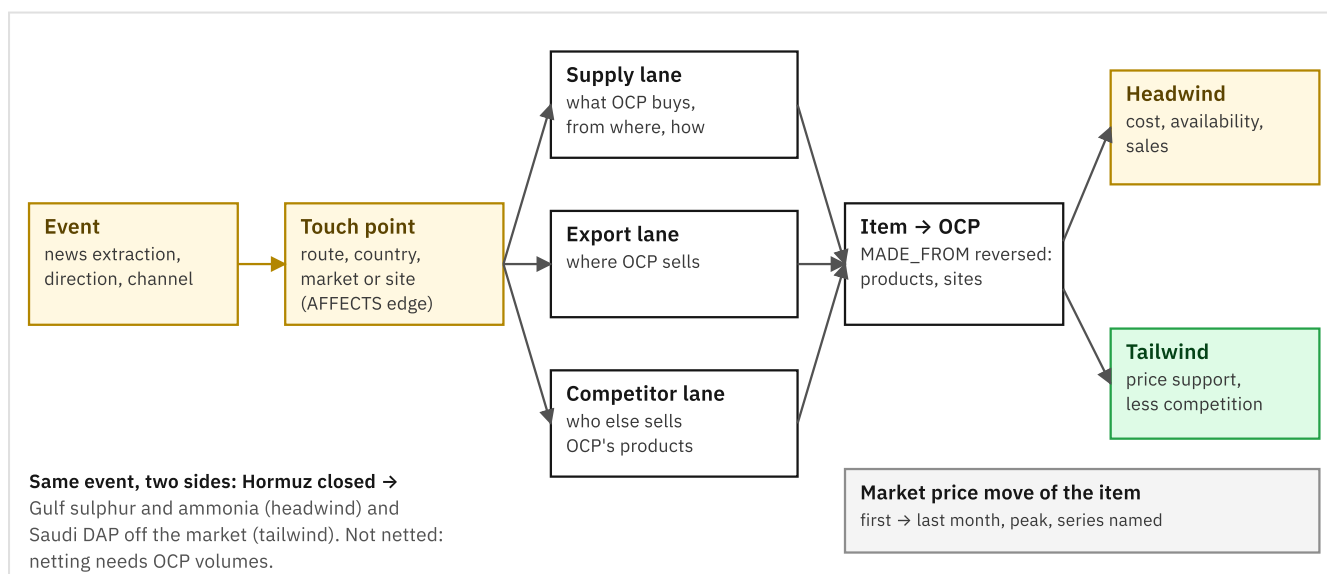


Figure 7. From an event to OCP's products and sites, with the effect named and the market price move attached.

Link	What happened	Effect for OCP
Supply lane (OCP buys)	origin or route disrupted	headwind: availability, input cost
Export lane (OCP sells)	route or destination disrupted	headwind: sales delayed or costlier
Competitor lane	competitor's origin or route disrupted, export restriction	tailwind: price support
Input market / product market	price up	headwind for inputs, tailwind for products
OCP site	outage, strike, accident	headwind on what the site makes

On the real graph, asked "OCP a beaucoup perdu cette année, pourquoi ?", the agent said first that it cannot confirm or size a loss, then listed dated incidents with article ids: Hormuz closure and sulphur backlogs (headwinds on sulphur and ammonia reaching Jorf Lasfar and Safi), DAP and phosphoric acid prices up about 28-29% from January to June (tailwinds), the partial reopening and the June peace deal. Reading that answer exposed two flaws now fixed: news that merely mentioned Morocco flagged OCP's export lanes, and the cost side had no prices while the selling side had them. The rule the agent follows: never conclude that selling prices offset costs when costs have no price.

6.5 Branch submissions with no template: structure discovery and the P&L engine

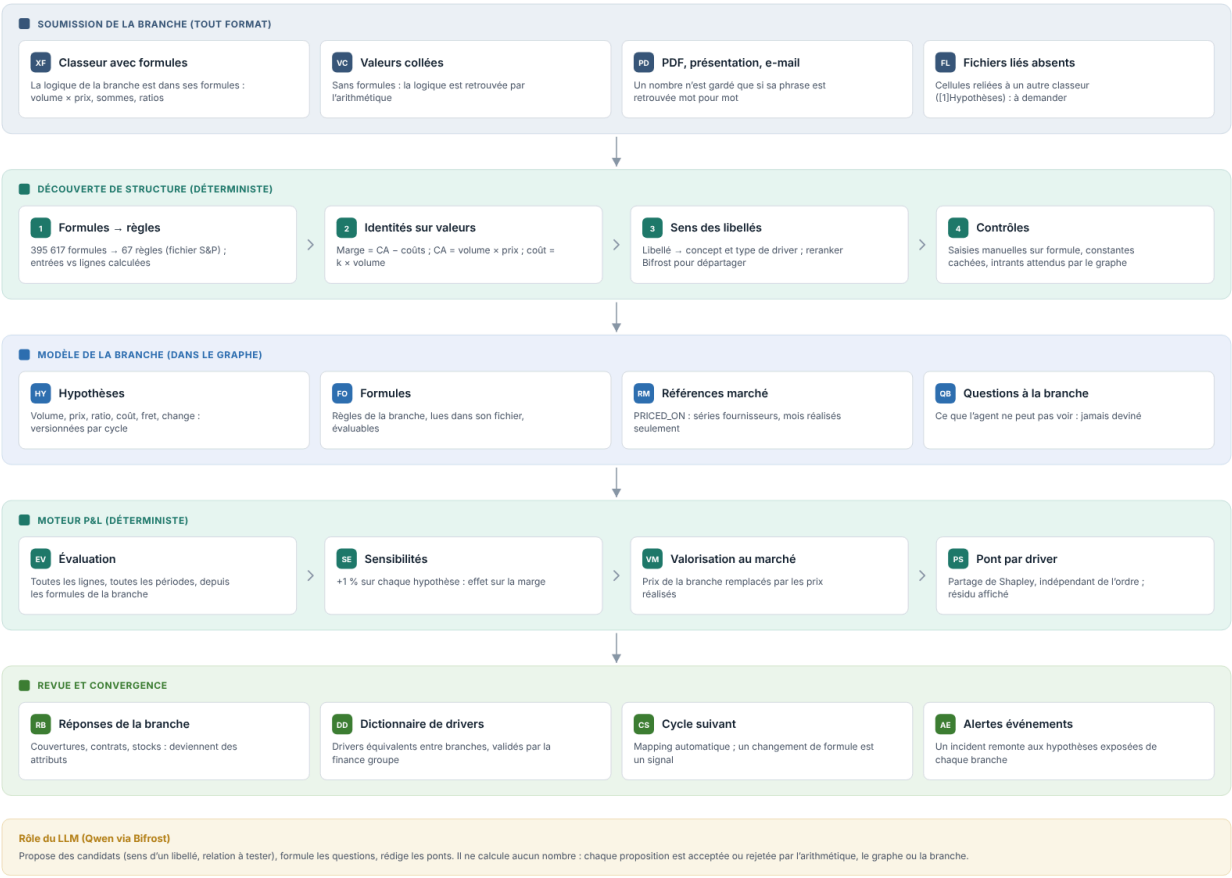
Branches send their own spreadsheets, so the first job is to discover their structure, most reliable layer first, each checked by code:

1. **Formulas → rules.** Cell references become row labels and generalise across rows and columns: inputs are separated from computed lines.
2. **Identities on pasted values.** Sums, revenue = $k \times \text{volume} \times \text{price}$, constant ratios (unit costs, consumption), flat lines and constant growth, tested on the numbers.
3. **Meaning.** Labels → concepts and driver types (price, volume, consumption, freight, FX, fixed cost) through the graph's terms; the reranker breaks ties.
4. **Questions back to the branch:** numbers typed over a formula, constants hidden inside formulas, links to files we do not have, inputs the graph expects but the file lacks.

The engine then evaluates the branch's own rules, computes sensitivities, marks price assumptions to provider actuals (actual months only) and splits the change with an exact Shapley bridge plus an unexplained residual. On a made-up branch file (*illustrative numbers, not OCP data*) with real Argus actuals: the hidden sulphur cost of 66 \$/t DAP was recovered as $0.40 \text{ t/t} \times 165 \text{ $/t}$; the margin moved from 1,062.6 to 1,083.7 M\$ when marked to market, split into +49.0 from the DAP price and -27.9 from ammonia, with nothing unexplained; sulphur had no market series and was kept as submitted, which the report says.

Soumission d'une branche — découverte de structure et P&L

Pas de modèle imposé : l'agent retrouve la logique de chaque fichier, le moteur calcule, la branche confirme, le modèle converge



Exemple outillé : scripts/discover_submission.py (branche fictive, prix réalisés Argus) ; chiffres illustratifs, pas des données OCP.

Figure 8. Branch submission discovery and P&L flow (French original, Figma-ready SVG in docs/figures).

7 Platform

Part	Technology	Role
Graph	Neo4j 5 Community	All layers in one database, namespaced by label
Market data	Postgres (existing market-intel volume)	Article text, metrics, referentials
API	FastAPI	Agent, workbooks, specs, variance, diagnoses, entities, news
UI	Streamlit, IBM Carbon theme (DESIGN.md)	Tabs: Ask the agent, Variance (with review queue), News & prices, Entities, Workbooks, Spec review, Eval
Run	podman compose	neo4j, market-intel-db, one-shot loader, api, ui
Quality	pytest (42 tests, no database needed)	Executor, discovery, P&L engine, variance, exposure, agents with a fake model

Checks for operators: `check_azure.py` (DNS, key, deployments, tool calling), `check_gateway.py` (Bifrost), `check_driver_series.py` (which price routes have data), `smoke_variance.py`, `discover_submission.py`. Errors from the LLM provider reach the UI as a message with the fix, not a stack trace.

8 Limits and risks

- **No OCP numbers.** The graph has no OCP volumes, costs or results. The system shows which links were hit and how market prices moved; it cannot say how much OCP lost or gained, or net headwinds against tailwinds.
- **Unconfirmed structure.** The ontology, the 20 supply-chain lanes and the input ratios are seeded from public knowledge or are illustrative. Wrong lanes produce wrong exposures; they are labelled as such in every answer.
- **Price series.** Driver series are first guesses at routes; where the configured route has no data the tool falls back to the most quoted route and names it. Morocco fob DAP is thin (1-8 points a month).
- **Evaluation.** Variance results come from a synthetic market; extraction results from 8 sheets. Real labelled episodes are needed.
- **Licensing and hosting.** Argus, CRU and S&P content is licensed; what may be stored, and whether data may leave the network for a cloud model, must be confirmed. The Bifrost/Qwen path keeps everything on premises.
- **Model quality.** Small models write plausible but wrong specs; the checks catch most of it, and human approval stays in the loop.

9 Next steps and what is needed from OCP

Step	Needs
Size gains and losses per driver	Monthly volumes and results per product or branch (actual and budget)
Confirm the supply chain	Procurement: sulphur and ammonia origins, shares, routes. Sales: markets and shares per product
Dollar impact per driver	Consumption ratios (t sulphur, ammonia, rock per t of product)
Load branch models into the graph	Submission, Assumption, Formula nodes; a UI tab with lines, questions and the bridge

Step	Needs
Watcher	On a new incident, traverse to exposed assumptions of every branch and run the engine on a calibrated scenario
Vessel tracker extractor	Measured shipments through Hormuz and the Red Sea instead of assumed lanes
Market and business specialists	Forecast series, the formula DAG, the group driver dictionary
Real evaluation	Labelled episodes with controllers; the 2026 Hormuz episode first

A Architecture figures (French originals)

Drawn earlier in the ANAPEC diagram style; editable SVGs in `docs/figures/` .

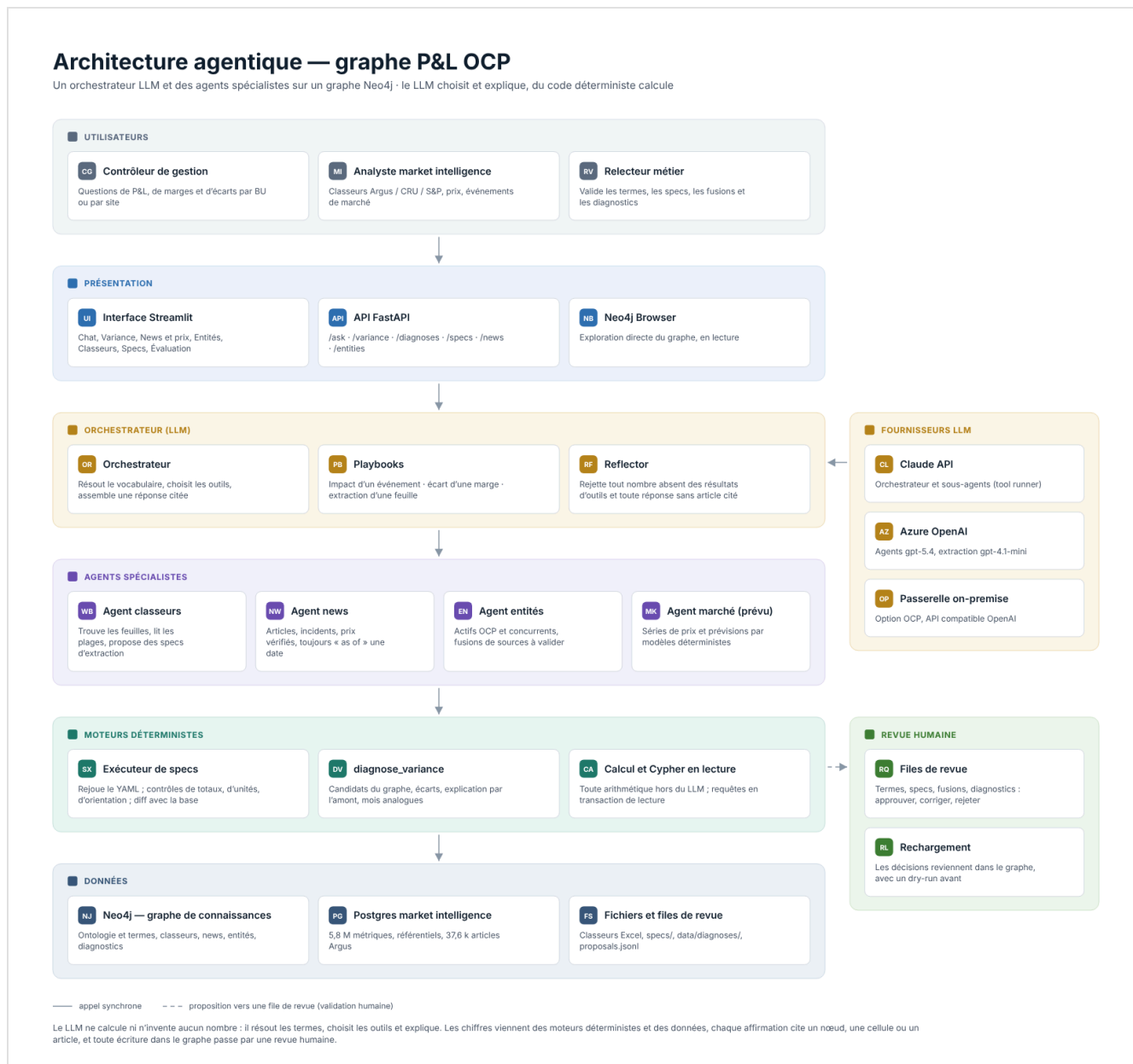


Figure A1. Architecture agentique.

Flux de données — des sources au graphe

Chargements idempotents · étapes LLM manuelles et mises en cache · toute requête temporelle est « as of » une date de publication



Restitution et revue (sortant)
Réponses citées, specs d'extraction et diagnostics de variance partent en file de revue (specs/, data/diagnoses/, proposals.jsonl) ; une fois validés, ils reviennent dans le graphe au prochain chargement.

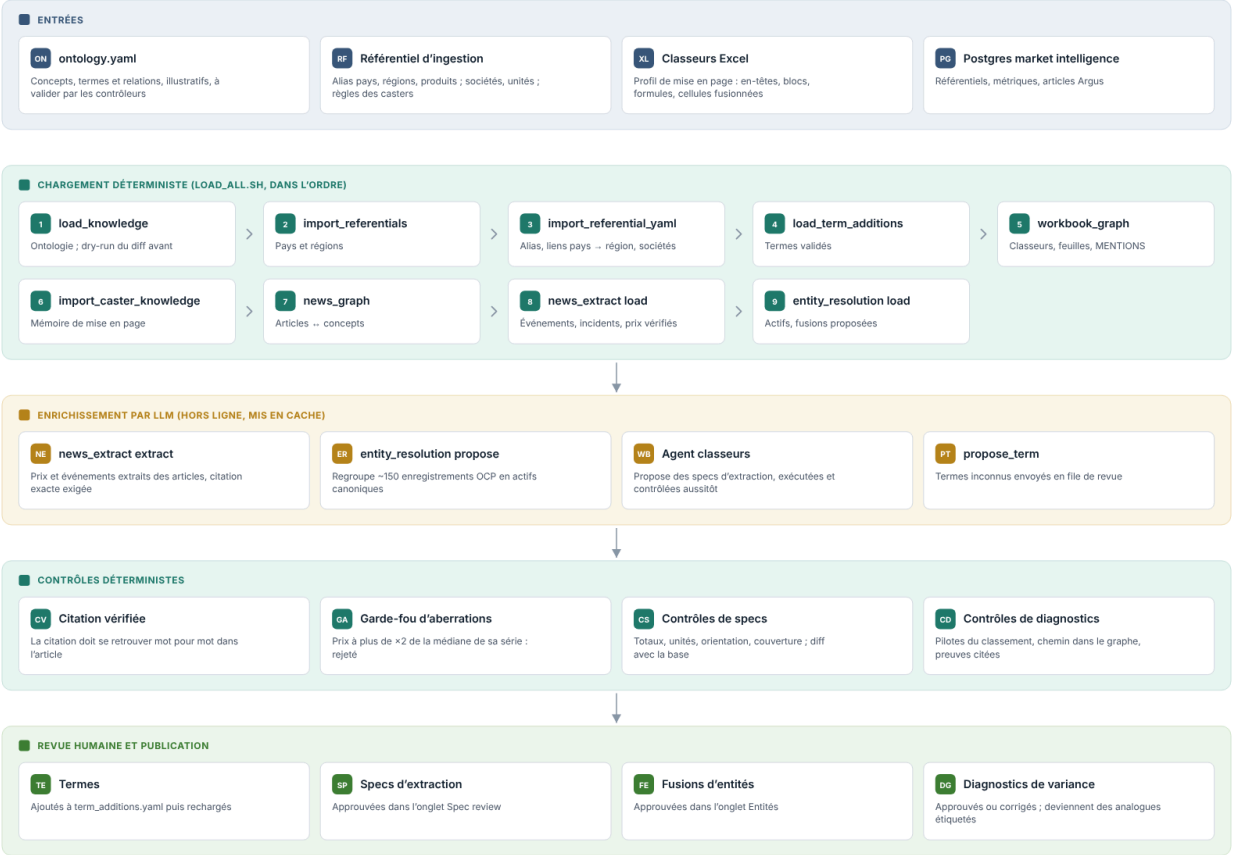
Mécanismes transverses de tous les flux
load_all.sh rejouable · dry-run de l'ontologie avant rechargement · requêtes « as of » sans fuite d'information future · étapes LLM lancées à la main et mises en cache · lecture seule pour les agents.

— chargement déterministe — étape avec extraction LLM vérifiée
Les volumes cités sont ceux de l'exploration du 25-09-2026 ; les routes de prix utilisées par le diagnostic sont à vérifier avec scripts/check_driver_series.py.

Figure A2. Flux de données.

Création du graphe — processus de chargement

Ordre de load_all.sh · le LLM enrichit hors ligne, des contrôles déterministes filtrent, un humain valide avant publication



Paramétrage : ontology.yaml · term_additions.yaml · driver_series.yaml · env (fournisseur LLM, NEO4J_URI, MARKET_INTEL_DSN, EXCEL_DIR)

Figure A3. Création du graphe.

B Where things live in the repository

Path	Content
knowledge/ontology.yaml · term_additions.yaml · supply_chain.yaml	Curated concepts, approved terms, supply-chain lanes
knowledge/driver_series.yaml · market_refs.yaml	Which price series stands for each driver; market references
src/agent.py	Orchestrator, specialists, Reflector, providers
src/excel_tools.py · spec_executor.py · formula_graph.py · discovery.py	Workbook tools, spec replay and checks, formulas, structure discovery
src/variance.py · exposure.py · pnl.py · market.py	Deterministic engines
src/news_*.py · entity_*.py · load_*.py · import_*.py	News layer, entities, loaders
docs/01-11	Vision, research notes, roadmap, findings, prior art, sub-agents, variance, sample files, P&L agent, exposure